

Reflection Paper

1. What I implemented

used the cleaned car dataset (clean_car_dataset.csv) to predict car price. I separated the data into features (every column except Price and LogPrice) and the target (Price), then split it into 80% training data and 20% testing data using train_test_split with random_state=42 so the results would be reproducible.

I trained two models required by the assignment: a LinearRegression model and a RandomForestRegressor with n_estimators=100 and random_state=42, both fit on the same training data. I also trained a DecisionTreeRegressor as an additional comparison, since it helped me understand overfitting more concretely. I wrote a single helper function, metrics_fun, to compute and print MAE, MSE, RMSE, and R² for any model so I could evaluate all three consistently, and I ran a sanity check by picking one row from the test set and comparing the actual price to each model's prediction.

2. Comparison of Models

In the sanity check, the actual price of the selected car was 1500. Random Forest and Decision Tree both predicted exactly 1500, matching the actual value perfectly, while Linear Regression predicted 2857, overestimating the price by almost double.

Actual price	Linear Regression	Random Forest	Decision Tree
1500	2857	1500	1500

On this single example, Random Forest gave the more realistic result. Linear Regression assumes a straight-line, additive relationship between features and price, so when a car has an unusual combination of features (for example, a low price relative to its other attributes), the linear model cannot adjust for that specific case the way a tree-based model can. Random Forest and Decision Tree can split the data into smaller groups based on feature values and make a more localized prediction, which is why they matched the actual price exactly for this particular car. That said, I know one sample is not enough to judge a model on its own, which is why the aggregate metrics in section 4 matter more than this single comparison.

3. Understanding Random Forest

a Random Forest is an ensemble model made up of many individual Decision Trees working together rather than relying on a single tree. Each tree in the forest is trained on a random subset of the training data (sampled with replacement, known as bootstrapping) and is also only allowed to consider a random subset of features at each split. This means every tree in the forest ends up slightly different from the others, since each one sees a different slice of the data and a different set of possible features to split on.

When making a prediction, Random Forest passes the input through every tree in the forest and then averages all of their individual predictions (for regression tasks) to produce the final output. This

averaging step is the key idea: a single Decision Tree can easily overfit and memorize noise in the training data, but when many such trees are combined and their predictions averaged, the individual errors and quirks of each tree tend to cancel out. The forest as a whole ends up more stable and generalizes better to unseen data than any single tree would on its own.

4. Metrics Discussion

Model	MAE	RMSE	MSE	R ²
Linear Regression	1428	1938	3,755,299	0.44
Random Forest	1213	2209	4,877,778	0.27
Decision Tree	1394	2574	6,626,224	0.00

The two metrics tell partly different stories here. Random Forest had the lowest MAE (1213, compared to 1428 for Linear Regression), meaning that on average its predictions were closer to the actual price across the test set. However, Linear Regression had the higher R² (0.44 versus 0.27 for Random Forest), meaning Linear Regression explained more of the overall variance in price.

This combination tells me that Random Forest is typically close to the correct price but occasionally makes a few large mistakes, since R² is based on squared error and is much more sensitive to large individual errors than MAE is. A handful of badly mispredicted cars (likely ones with rare feature combinations the forest had few similar training examples for) can pull R² down sharply even while most predictions remain accurate. Linear Regression, on the other hand, appears to be consistently mediocre rather than occasionally very wrong, which keeps its R² comparatively higher despite its worse average error. The Decision Tree performed worst overall, with an R² of 0.00 despite achieving a perfect R² of 1.0 on the training data, which is a clear case of overfitting: the tree memorized the training set instead of learning patterns that generalize to new cars.

5. My Findings

Overall, I would lean toward Random Forest for this car price prediction task, mainly because of its lower MAE and its strong performance in the sanity check, where it predicted the exact price for the sample car. Its ability to capture non-linear relationships and interactions between features (for example, how mileage might matter differently depending on the brand) makes it better suited to a problem like car pricing, where the real relationships are unlikely to be purely linear.

At the same time, the gap between Random Forest's R² and its MAE is a signal worth taking seriously rather than ignoring: it suggests the model is doing well on most cars but struggling on a smaller number of edge cases, possibly underrepresented brands, very old or very new cars, or unusual feature combinations with limited training examples. If I were continuing this project, I would want to investigate which specific test cases produce the largest errors for Random Forest, since improving those outliers, perhaps through more training data, additional features, or hyperparameter tuning, would likely raise its R² closer to a level that reflects how strong its average-case performance already is.